

Tu primera API REST con Spring Boot

Clase de taller: el profesor crea desde cero un proyecto Spring Boot en IntelliJ y escribe un conversor de temperaturas como API REST. Da dos de las seis conversiones en vivo y deja las otras cuatro para el grupo. Es la clase donde la teoría de SOAP y REST de la sesión anterior se vuelve código que corre.

El dispositivo de la página es la **URL de la API**. Cada pedazo se pinta del color de la línea de código de donde sale: **puerto** **ruta base** **método** **parámetro**. Los botones con ? explican los términos que dio por sabidos. Hay seis ejercicios intercalados.

Lo que salió de esta clase

≈32:10

Tarea: terminar el **conversor de temperaturas**. En clase hizo Celsius a Fahrenheit y Celsius a Kelvin. Faltan **cuatro**: F a C, F a K, K a C y K a F.

40:20

Revisión: "lo demás lo revisamos mañana en el laboratorio". Terminar el ejercicio antes de llegar.

≈48:15

Por confirmar: mencionó "la fecha del 17 de septiembre" al hablar de altas y bajas. No queda claro a qué se refiere.

≈49:50

Pendiente administrativo: el laboratorio de mañana (jueves 10) es en el **Laboratorio 3**, no en el de siempre.

i

Sobre esta transcripción. Es la versión ya limpia de la grabación, con marcas de tiempo solo en algunos momentos. Los bloques que no traen marca propia van dentro del bloque más cercano. Hay tres tropiezos del profesor, o de la transcripción, que marco donde aparecen y que conviene no copiar tal cual: la versión de Java (bloque 1), el nombre del endpoint de Kelvin (bloque 4) y lo que lee en la salida de `jq` (bloque 5).

BLOQUE 1 · MINUTO 00:00 A 07:35

Crear el proyecto en el IDE

Empieza con el asistente de proyecto nuevo de IntelliJ, que es en el fondo Spring Initializr. Va campo por campo y dice qué elegir.

*"Si ustedes quieren crear el repositorio Git de una vez, le pican a la casilla; si no, lo pueden hacer a manita desde terminal, se meten a la carpeta y le dan un **git init** y san se acabó."*

CAMPO	QUÉ DIJO	VALOR EN CLASE
Lenguaje	"Vamos a agarrar Java", ya viene marcado.	Java
Tipo	Maven, ya viene marcado.	Maven
Group ID	"Pónganle el que ustedes quieran." Lo dejó "por puro gusto".	com.ipn.mx
Artifact ID	Cómo se llama lo que desarrollas. Normalmente toma el nombre del proyecto y se puede cambiar.	ApiSimple
Package	Quitó el sufijo que ponía por defecto y dejó <code>com.ipn.mx</code> . Al nombre le puso <code>ApiSimple7CM2</code> .	com.ipn.mx
JDK	El que tengas instalado. Él tiene el 26. Si no tienes ninguno, el asistente puede descargar uno.	26
Versión de Java	"Lo mínimo que necesitamos para trabajar es la 17, es lo mínimo que se requiere de soporte."	≥ 17
Empaquetado	Solo JAR o WAR. Se queda con JAR.	JAR
Configuración	Acepta <code>application.properties</code> o <code>application.yml</code> . Se queda con properties.	properties



Tropiezo en la versión de Java. Al explicarlo dijo algo como "tienes un JDK 25, pues ponen 26". Eso no puede ser: la versión de Java del proyecto no puede pasar de la del JDK instalado. Lo natural es al revés, y su regla clara es que el piso es 17. Yo lo tomo como un lapsus.



¿Qué son groupId, artifactId y Maven?

Maven es la herramienta que descarga las dependencias y construye el proyecto a partir del archivo `pom.xml`. El **groupId** identifica a quien lo hace, normalmente con el dominio al revés (`com.ipn.mx`). El **artifactId** es el nombre del proyecto. Juntos son la "dirección" con la que se identifica el proyecto.

JAR, WAR, EAR y RAR

Explicó los cuatro tipos de empaquetado de Java:

- **.jar** (Java Archive): lo típico para aplicaciones de escritorio y de consola, un ejecutable.
- **.war** (Web Application Archive): para web. Se mete en la carpeta correspondiente de un Tomcat, un TomEE, un GlassFish o un Payara y se carga.

- **.ear** (Enterprise Application Archive): para módulos empresariales con EJB. Si no se separan los proyectos por parte, todo se empaqueta en un JAR o en un WAR "y tampoco pasa nada".
- **.rar** (Resource Adapter Archive): "no es el ZIP tradicional", vive en el ecosistema empresarial y "rara vez se utiliza". Es para recursos.

*"Por definición aquí solo tenemos dos archivos: JAR o WAR. ¿Funcionan muy similar? Sí y no. El JAR lo que trae incrustada es una versión muy minimalista de un Tomcat, **Tomcat embebido**. Por eso puede funcionar."*

– ¿Qué es EJB?

Enterprise JavaBeans: la tecnología empresarial de Java para componentes de lógica de negocio que corren dentro de un servidor de aplicaciones. El profesor solo lo nombró para explicar cuándo tiene sentido un EAR.

properties o **yaml**

Los dos hacen lo mismo. El **YAML** se organiza por sangría: empiezas con el elemento principal y vas bajando según la propiedad. Comentó que a quien viene de Python le resulta natural, y que a él le cuesta. Aquí usa **properties**.

– ¿Cómo se ve la misma propiedad en cada formato?

```
# application.properties
server.port=8082

# application.yml
server:
  port: 8082
```

En **YAML** cada nivel del nombre baja una línea y una sangría. Un espacio de más o de menos cambia a qué elemento pertenece la clave. El profesor lo llamó "tabuladores", pero en **YAML** la sangría debe ser con espacios.

Las dependencias

Fue enumerando lo que necesitaría el proyecto completo: **Spring Web**, **Validation**, **Spring Data JPA**, el conector de **MySQL** y **DevTools**. Contaba "1, 2, 3, 4, 5" y se preguntó si le faltaba uno.

Alumno: "¿Security?" **Profesor:** "No, ahorita no. Eso vamos a dejarlo para después. Pero sí, el Security sí, eventualmente sí, pero me parece

*que el que trae este es la versión 6 todavía. La versión actual es la 7. [...]
Vamos a la documentación y tomamos la dependencia y se la pegamos
al pom.xml."*

Para hoy se queda solo con dos:

- **DevTools**, "nada más para que esté jugando" (reinicio automático al guardar).
- **Web**, que es lo que permite hacer la API.

– ¿Para qué sirven Validation, Data JPA y DevTools?

- **Spring Web**: trae Spring MVC y un Tomcat embebido para recibir peticiones HTTP.
- **Validation**: comprobar que los datos que llegan cumplan reglas (obligatorio, rango, formato).
- **Spring Data JPA**: hablar con la base de datos sin escribir el SQL a mano.
- **MySQL**: el controlador (driver) para conectarse a esa base.
- **DevTools**: reinicia la aplicación sola cuando guardas un cambio.

Solo se usaron Web y DevTools en esta clase. Lombok se le olvidó agregarlo; salió después.

Los campos y el empaquetado

Cinco preguntas sobre lo que eligió en el asistente y por qué. Contesta las cinco y dale a **Revisar**.

¿Qué Group ID usó?

com.ipn.mx

com.spring.api

ApiSimple7CM2

¿Cuál es la versión mínima de Java para trabajar?

11

17

21

Una aplicación web que se va a meter en la carpeta de un Tomcat o un Payara, ¿en qué empaquetado va?

.jar

.war

.rar

¿Por qué el .jar de un proyecto Spring puede correr solo, sin un servidor aparte?

Trae un Tomcat embebido, minimalista

Se convierte solo en .war

Windows lo ejecuta directo

¿Qué dependencias marcó al final para la clase de hoy?

DevTools y Web

Data JPA y MySQL

Security y Validation

Revisar

Reiniciar

BLOQUE 2 · MINUTO 07:35 A 13:20

Lo que generó el asistente, y tres tipos de servidor

Con el proyecto creado, recorrió los archivos y de ahí saltó a una explicación de servidores que desemboca en el único cambio de configuración del día.

Recomendó abrir `HELP.md` "en algún momento". Apunta a la documentación oficial de Maven, de Spring y de Web, y trae guías como "Construye un servicio REST" o "Construye una aplicación web con MVC". Sobre la estructura: como el proyecto es en teoría web, genera las carpetas para un proyecto de ese estilo, aunque no se haya agregado Spring MVC explícitamente.

En `src/main/resources` hay dos carpetas que hoy no se tocan:

- `static/`: recursos estáticos, como estilos e imágenes.
- `templates/`: los HTML, cuando el proyecto pinta páginas. Funciona con un framework extra, **Thymeleaf**, que hace de motor de plantillas.

*"Nosotros no lo vamos a tocar, solo vamos a modificar algo de aquí: del **application.properties**. Y lo único que le vamos a poner por ahorita va a ser un **server.port=8082**. ¿Por qué? Por puro gusto."*

– ¿Qué es Thymeleaf?

Un motor de plantillas para Java: escribes HTML normal con marcas especiales y el servidor las rellena con datos antes de mandar la página. Es la opción para páginas del lado del servidor, distinta de una API que devuelve JSON. El profesor solo lo nombró para explicar la carpeta `templates`.

Web Server, Web Container y Application Server

Para explicar por qué cambia el puerto, hizo un recorrido por los servidores del ecosistema Java. Tres capas, cada una con su especialidad:

WEB SERVER

Apache HTTP Server

Entiende HTML, CSS, JavaScript y recursos estáticos: audio, video, imágenes.

WEB CONTAINER • el de esta clase

Tomcat (aquí, embebido)

Servlets, JSP y frameworks web de Java: Spring MVC, Jakarta Faces. Puerto 8080 por defecto.

APPLICATION SERVER

GlassFish • Payara • WebLogic • WebSphere • WildFly (JBoss)

Servicios, data source, pool de conexiones. Puerto 8080 por defecto.

La idea de "capas físicas": una carcasa web simple pide el primero; la parte que procesa Java para web, el segundo; la que maneja servicios y pool de conexiones, el tercero.

"Y esos dos deben tener un puerto diferente. Entonces nosotros vamos a jugar con una versión minimalista del Tomcat, lo vamos a meter en el 8082."

– ¿Qué es un pool de conexiones y un data source?

Abrir una conexión a la base de datos cuesta. Un **pool** mantiene un grupo de conexiones ya abiertas y se las presta a quien las pida. El **data source** es el objeto configurado que

sabe a qué base conectarse y entrega esas conexiones. Los servidores de aplicaciones los administran por ti.

EJERCICIO 2 · SERVIDORES Y PUERTOS

¿Qué hace cada uno?

Cuatro preguntas sobre la diferencia entre los tres tipos de servidor y el puerto.

¿Qué entiende un **Web Server** como el Apache HTTP Server?

HTML, CSS, JavaScript y recursos estáticos

Servlets y JSP

Data sources y pools de conexiones

¿Y un **Web Container** como Tomcat?

Solo archivos HTML

Tecnología Java para web: Servlets, JSP, Spring MVC, Jakarta Faces

Bases de datos

¿Cuál de estos es un **Application Server**?

Nginx

Payara

Insomnia

Tomcat y un servidor de aplicaciones usan por defecto el mismo puerto. ¿Cuál?

80

8080

3306

Revisar

Reiniciar

BLOQUE 3 · MINUTO 13:20 A 32:10

El DTO, el controlador y el primer endpoint

Es el bloque más largo y el centro de la clase. Va de tres paquetes vacíos a un método que responde en el navegador.

El DTO, como **record**

"Nos dicen que un DTO es una clase Java simple que probablemente implemente la interfaz de serialización, que tenga atributos privados a la clase, un constructor con argumento cero... ¡Ya me acordé qué otra

cosa me hacía falta! Se me olvidó poner Lombok para no escribir cosas."

En lugar de una clase común, usa un `record`, "porque esta cosa no va a cambiar". Lo llama `ConversionResponse`, lo quiere "para respuesta", y lo crea en un paquete `dto`:

```
package com.ipn.mx.dto;

public record ConversionResponse(
    Double valorOriginal,
    String unidadOriginal,
    Double valorRespuesta,
    String unidadRespuesta
) {}
```

Podría ser una clase, con getters y setters para cada campo. Este es "literal, un objeto que funciona simplemente pasándole valores a los atributos y recuperando los valores". Es **immutable**, y por eso le queda bien a un DTO.

— ¿Qué es un `record` y qué gano con él?

Una forma corta de declarar una clase de datos. Escribes solo los campos entre paréntesis y Java genera el constructor, los métodos de lectura, `equals`, `hashCode` y `toString`. Sus campos no se pueden cambiar después de creado el objeto (es immutable), y no hay setters.

Para un DTO de respuesta es justo lo que hace falta: se arma una vez, se manda y no se toca. Necesita Java 16 o superior, por eso conecta con el mínimo de 17 de antes.

El controlador

Nuevo paquete `controller`, nueva clase `TemperaturaController`. Dos anotaciones sobre la clase:

- `@RestController`: "porque vamos a crear un API". Al ser REST, "debería de permitirnos utilizar todos los verbos del protocolo".
- `@RequestMapping("/api/temperatura")`: "tengo que encontrar la forma para llegar a este cuate". Es la ruta base de todo lo que tenga dentro.

```
@RestController
@RequestMapping("/api/temperatura")
public class TemperaturaController {

}
```


Con eso ya se sabe cómo empieza cualquier URL de la API. Y aquí llega la pieza que ordena la clase: **cada pedazo de la URL sale de una línea concreta.**

`http://localhost:8082 /api/temperatura /convertir-a-fahrenheit ?valor=10`

- **Servidor y puerto.** El puerto viene de `server.port` en `application.properties`. El host es donde corre el servidor: "mi localhost, mi 127.0.0.1, mi 192.168.x.x", o el nombre si hay DNS.
- **Ruta base.** El `@RequestMapping` de la clase.
- **Nombre del método.** El `@GetMapping` de cada método.
- **Parámetro.** El `@RequestParam`, que llega después del signo de interrogación.

"Cualquier cosa que nosotros vayamos a solicitar a la URL tenemos que pegar esto de arranque, hasta ahí. ¿De dónde sale? De este archivito. [...] Y el resto lo obtenemos justo de la definición del RequestMapping en el controller."

— ¿Qué es `localhost` y por qué da lo mismo que `127.0.0.1` ?

`localhost` es el nombre que tu propia máquina se da a sí misma; se traduce a la dirección `127.0.0.1`. Si el servidor corre en la misma computadora que el navegador, apuntas ahí. Si corriera en otra, pondrías su IP (como `192.168.x.x` en una red local) o su nombre de dominio.

El primer método: Celsius a Fahrenheit

Solo hace falta un verbo, `@GetMapping`, porque "no vamos a insertar, no vamos a actualizar, no vamos a eliminar". La idea es: "método fulanito, te paso un argumento, opera con él y devuélveme un resultado".

El método es público, devuelve un `ResponseEntity` de `ConversionResponse` y se llama `celsiusToFahrenheit`. Recibe el parámetro `valor` como `Double`, guardado en una variable `celsius`:

```
@GetMapping("/convertir-a-fahrenheit")
public ResponseEntity<ConversionResponse> celsiusToFahrenheit(
    @RequestParam(name = "valor") Double celsius
) {
    Double fahrenheit = (celsius * 9 / 5) + 32;
    ConversionResponse response = new ConversionResponse(celsius, "°C", fahrenheit, "°F")
    return ResponseEntity.ok(response);
}
```

Los argumentos del constructor van en el orden del record: valor original (`celsius`), unidad original (`"°C"`), valor de la respuesta (`fahrenheit`) y unidad de la respuesta (`"°F"`).

– ¿Qué hace `@RequestParam` ?

Le dice a Spring que tome un valor de los parámetros de la URL, los que van después del `?`. Con `name = "valor"` busca `?valor=...` y lo convierte al tipo que declares, aquí `Double`. El nombre de la variable de Java (`celsius`) puede ser distinto del nombre del parámetro (`valor`).

*"Tomen ustedes una decisión: ¿le van a poner nombre o simplemente se van a referir a esto? Cada método que ustedes mapeen puede llevar un nombre en particular. [...] Como son puros GET, sí hay que diferenciarlos: eventualmente o reciben un argumento o tienen un nombre. En este caso lo más sano es **ponerle un nombre.**"*

– ¿Por qué dos `@GetMapping` sin nombre chocarían?

Si dos métodos responden al mismo verbo y a la misma ruta, Spring no sabe a cuál mandar la petición y falla al arrancar. Con un nombre distinto en cada `@GetMapping` (`/convertir-a-fahrenheit`, `/convertir-celsius-a-kelvin`) cada uno tiene su propia ruta.

i `ResponseEntity` desde el primer método. En la clase anterior el profesor dijo que la entidad no debe regresarse sola, sino dentro de un `ResponseEntity`. Aquí es lo primero que hace. `ResponseEntity.ok(response)` devuelve el objeto con código 200.

La URL, línea por línea

Cada trozo de `http://localhost:8082/api/temperatura/convertir-a-fahrenheit?valor=10` viene de una línea distinta. Une cada uno con su origen.

`:8082` del `localhost:8082`

`application.properties`

`@RequestMapping` (clase)

`@GetMapping` (método)

`@RequestParam`

`/api/temperatura`

`application.properties`

`@RequestMapping` (clase)

`@GetMapping` (método)

`@RequestParam`

`/convertir-a-fahrenheit`

`application.properties`

`@RequestMapping` (clase)

`@GetMapping` (método)

`@RequestParam`

`?valor=10`

`application.properties`

`@RequestMapping` (clase)

`@GetMapping` (método)

`@RequestParam`

Revisar

Reiniciar

EJERCICIO 4 · ARMA EL MÉTODO

`celsiusToFahrenheit`, en orden

Toca las líneas en el orden en que las escribíó. Si te equivocas te doy una pista; toca una línea ya puesta para quitarla.

Aquí se va armando el método...

```
}
```

```
ConversionResponse response = new ConversionResponse(celsius, "°C", fahrenheit, "°F");
```

```
return ResponseEntity.ok(response); @GetMapping("/convertir-a-fahrenheit")
```

```
Double fahrenheit = (celsius * 9 / 5) + 32;
```

```
public ResponseEntity<ConversionResponse> celsiusToFahrenheit(@RequestParam(name =  
"valor") Double celsius) {
```

Reiniciar

BLOQUE 4 · MINUTO 32:10 A 36:45

Probarlo, la tarea y Celsius a Kelvin en vivo

Corrió el proyecto, pegó la URL en el navegador y comprobó dos valores. Luego dejó el resto del conversor.

Pegó `http://localhost:8082/api/temperatura/convertir-a-fahrenheit?valor=10` en el navegador. Con 10 respondió 50, y con 20 respondió 68:

```
{  
  "valorOriginal": 20.0,  
  "unidadOriginal": "°C",  
  "valorRespuesta": 68.0,  
  "unidadRespuesta": "°F"  
}
```

"¿Estamos de acuerdo, jóvenes? Sencillito, ¿no?"

Terminar el conversor de temperaturas

Después de la prueba anunció cinco conversiones para el grupo (C a K, F a K, F a C, K a C y K a F), con "30 minutos para acabar". Enseguida hizo él la de **Celsius a Kelvin**, así que quedan **cuatro**. Lo demás se revisa "mañana en el laboratorio".

"Bueno, tienen 30 minutos para acabar [...]. Literal, todo el conversor. Eso es un API."

CONVERSIÓN	FÓRMULA	ESTADO
C → F	<code>(celsius * 9 / 5) + 32</code>	Hecha por el profesor
C → K	<code>celsius + 273.15</code>	Hecha por el profesor
F → C	<code>(fahrenheit - 32) * 5 / 9</code>	Tarea. Fórmula dicha en clase (bloque 12)
K → F	<code>(kelvin - 273.15) * 9 / 5 + 32</code>	Tarea. Fórmula dicha en clase (bloque 12)
F → K	<code>(fahrenheit - 32) * 5 / 9 + 273.15</code>	Tarea. No la dijo; sale de combinar F → C y C → K
K → C	<code>kelvin - 273.15</code>	Tarea. No la dijo; es la inversa de C → K

Celsius a Kelvin, copiando el método

Copió el método anterior y lo fue ajustando. "Ya rezongó" es el IDE marcando error cada vez que cambia algo a medias; se arregla al terminar de renombrar. Cambió la variable a `kelvin`, la operación a `celsius + 273.15` y la unidad de respuesta a `"K"`. Como ya no hay operación extra, "hasta le podemos quitar ese paréntesis".

```
@GetMapping("/convertir-celsius-a-kelvin")
public ResponseEntity<ConversionResponse> celsiusToKelvin(
    @RequestParam(name = "valor") Double celsius
) {
    Double kelvin = celsius + 273.15;
    ConversionResponse response = new ConversionResponse(celsius, "°C", kelvin, "K");
    return ResponseEntity.ok(response);
}
```

La prueba: `http://localhost:8082/api/temperatura/convertir-celsius-a-kelvin?valor=20` devolvió **293.15**. "Pues creo que sí hace su chamba, ¿no?"

! **El nombre del endpoint de Kelvin.** El bloque de código de la transcripción dice `/convertir-a-kelvin`, pero en voz alta lo renombra a `convertir-celsius-a-kelvin` y esa es la URL que prueba. Yo dejo la versión que se probó. Ojo con la diferencia de estilo: el de Fahrenheit se quedó como `convertir-a-fahrenheit`, sin "celsius". Él dijo que el nombre lo decide cada quien.

EJERCICIO 5 · HAZ LAS CUENTAS

¿Qué devolvería `valorRespuesta` ?

Escribe el número que el endpoint pondría en `valorRespuesta`. Vale con punto o coma decimal.

10 °C a Fahrenheit, con `/convertir-a-fahrenheit?valor=10`.

`valorRespuesta` =

Comprobar

20 °C a Kelvin, con `/convertir-celsius-a-kelvin?valor=20`.

`valorRespuesta` =

Comprobar

212 °F a Celsius, con la fórmula que dio al final.

`valorRespuesta` =

Comprobar

273.15 K a Fahrenheit, con la fórmula que dio al final.

`valorRespuesta` =

Comprobar

Reiniciar

BLOQUE 5 · MINUTO 36:45 A 39:20

curl y jq como cliente

Un compañero había comentado que lo ideal es probar con `curl`. El profesor lo mostró.

```
curl -s "http://localhost:8082/api/temperatura/convertir-a-fahrenheit?valor=10"
```

```
# con jq, para verlo con formato
```

```
curl -s "http://localhost:8082/api/temperatura/convertir-a-fahrenheit?valor=10" | jq
```

La primera versión devuelve el JSON tal cual, en una sola línea. Con un **pipe** (`|`) hacia `jq` se imprime "bonito", con encabezados y formato. También pidió disculpas por los tonos de colores de la terminal, que no se veían bien.

– ¿Qué son `curl`, el pipe y `jq` ?

- **curl** es un programa de línea de comandos para hacer peticiones HTTP. La opción `-s` (silencioso) oculta su barra de progreso.
- El **pipe** (`|`) toma la salida de un comando y se la pasa como entrada al siguiente.
- **jq** lee JSON y lo imprime con sangría y colores. Sin él la respuesta sale en una línea difícil de leer.

! **Tropiezo al leer la salida.** Al leer el resultado de `valor=10` dice "valorOriginal: 50". Es un lapsus: con 10 de entrada, `valorOriginal` es **10** y `valorRespuesta` es **50**.

Cierra el bloque avisando cuánto tiempo queda: "les quedan 26 minutos, tiempo suficiente para terminarlo".

BLOQUE 6 · MINUTO 39:20 A 40:20

Lo que ya se vio sin nombrarlo

Con unas diapositivas, junta en un minuto los conceptos que la práctica tocó sin decirlos.

- **El verbo GET.** "Ya estamos revisando el primero."
- **Códigos de respuesta.** "A lo mejor no lo visualizamos, pero esto nos está devolviendo un 200 (`HttpStatus.OK`)."
- **Endpoints.** "Construimos el endpoint con el que estamos trabajando."

i **Conexión con la clase 7.** El 200 es el primero de los códigos que el profesor puso en `@ResponseStatus` allá (OK, CREATED, NO_CONTENT). Y los códigos HTTP son la tarea que arrastra desde la clase 5.

"Señores, lo demás lo revisamos mañana en el laboratorio, pero terminen este ejercicio, ¿va?"

BLOQUE 7 · MINUTO 40:20 A 43:30

Probar en una herramienta gráfica

Un cliente con interfaz en lugar de navegador o terminal. El profesor lo ofrece como opción: "ya ustedes elijan con qué trabajar".

Pasos que siguió:

1. Crear una **colección**, que sirve solo para organizar. La llamó **Conversor Temperaturas**.
2. Crear el primer **request**, de tipo `GET`, y pegarle la URL.
3. Pasar el parámetro por la URL, o bien agregarlo en la sección de **Query Params**: `Name = valor`, `Value = 50`.
4. Darle **Send**.

Observó algo útil: si quitas `?valor=...` de la URL, funciona igual, porque el valor está en Query Params. La URL que se pega en la herramienta "no lleva el signo de interrogación", pero el *preview* de la URL lo vuelve a mostrar. La respuesta trae el **código 200 OK**, el tiempo y otros datos.

i **Cuál herramienta.** El profesor no la nombra en el texto, solo dice "con este". El encabezado de la transcripción habla de Postman o Insomnia. Los pasos aplican a las dos.

BLOQUE 8 · MINUTO 43:30 A 45:10

Qué pasa si quitas `server.port`

Una demostración corta de lo único que hace esa propiedad.

El `application.properties` trae por defecto `spring.application.name`, el nombre del proyecto. Lo único que se agregó fue `server.port`. Sin ella, el proyecto igual arranca, pero busca el puerto 8080.

Lo demostró con la herramienta gráfica: dejando la URL en 8082 y quitando la propiedad, el **Send** dice que no se puede conectar al servidor, porque Tomcat se apropió del 8080. Lo dejó de nuevo como estaba:

```
server.port=8082
```

"Es lo único que hace esa propiedad."

BLOQUE 9 · MINUTO 45:10 A 48:15

El mismo DTO, como clase con Lombok

Respondió a "¿y si quiero una clase en lugar de un record?": por supuesto que se puede.


```

package com.ipn.mx.dto;

import lombok.Data;
import lombok.NoArgsConstructor;
import lombok.AllArgsConstructor;
import lombok.Builder;
import java.io.Serializable;

@Data
@NoArgsConstructor
@AllArgsConstructor
@Builder
public class ConversionResponse implements Serializable {
    private Double valorOriginal;
    private String unidadOriginal;
    private Double valorRespuesta;
    private String unidadRespuesta;
}

```

El clásico `implements Serializable`, y con **Lombok**: `@Data`, `@NoArgsConstructor`, `@AllArgsConstructor` "y, ¿por qué no?, `@Builder`". Cuatro atributos privados, y ya. El controlador "ni siquiera cambiaría": debería funcionar igual.

– ¿Qué hace cada anotación de Lombok?

- `@Data`: genera getters, setters, `toString`, `equals` y `hashCode`.
- `@NoArgsConstructor`: el constructor sin argumentos.
- `@AllArgsConstructor`: el constructor con todos los campos.
- `@Builder`: permite armar el objeto paso a paso con el patrón Builder.

i Dos cosas que conectan, dichas por mí y no por el profesor. Primero: para usar esta versión hay que agregar la dependencia de Lombok al proyecto, y en el asistente del bloque 1 no se marcó. Con el `record` no hace falta. Segundo: `@Builder` es el patrón Builder que dejó de investigación en la clase 5 ("qué es, para qué sirve, con qué se come"). Aquí lo tienes en un ejemplo real.

EJERCICIO 6 · REPASO DE LA CLASE

Puerto, record, código de estado y herramienta gráfica

Cuatro preguntas sobre lo que sigue en pie de la clase.

¿Qué código HTTP devuelve `ResponseEntity.ok(response)` ?

200 (OK)

201 (CREATED)

204 (NO_CONTENT)

¿Por qué usó un `record` para `ConversionResponse` y no una clase común?

Porque no va a cambiar: es inmutable

Porque las clases no funcionan en Spring

Porque necesita más getters

Quitás `server.port` del `application.properties` y llamas a `localhost:8082`. ¿Qué pasa?

Responde igual

Error de conexión: el servidor se quedó en el 8080

Spring elige otro puerto libre

En la herramienta gráfica, ¿cómo mandas `valor=50` sin escribirlo en la URL?

En Body, con un JSON

En Query Params: Name = valor, Value = 50

No se puede, va siempre en la URL

Revisar

Reiniciar

BLOQUE 10 · MINUTO 48:15 A 55:40

Avisos de grupo y cambio de laboratorio

Rato de trabajo individual con avisos administrativos de por medio. No hay contenido técnico nuevo.

Contó a los presentes, comentó que "aguantaron hasta la fecha del 17 de septiembre" y preguntó por los alumnos a los que dieron de baja por no haber tomado una materia previa. Un alumno dijo que venía de movilidad y que ya lo iban a inscribir. Otra frase suya: sabe de "al menos tres que no han cursado la deserción".

- ! **La fecha del 17 de septiembre.** No queda claro qué se cumple ese día: puede ser el plazo de altas y bajas del grupo, pero no lo aclara. En el repo el 17 de septiembre (18:00) es la entrega de la investigación de HTTP, que es otra cosa. Si te afecta, pregúntale.

"Oigan, por cierto: mañana nos toca laboratorio, pero nos toca en el

Laboratorio 3. *Ya nos van a cambiar. Los jueves es en el 3, ¿vale?"*

PENDIENTE ADMINISTRATIVO ≈49:50 y cierre 64:47

Laboratorio del jueves 10, en el Laboratorio 3

Ahí se revisan los demás endpoints. Al cierre lo repitió: "mañana continuamos en el Laboratorio 3".

"Los jueves es en el 3, ¿vale?"

BLOQUE 11 · MINUTO 55:40 A 59:00

La documentación de Spring y el Initializr web

Cerró la parte técnica con dónde seguir aprendiendo y una forma de crear proyectos sin el IDE.

*"Que sea su documentación, que sea su **biblia** en esas noches de insomnio, cuando no tengan nada de ganas de hacer nada, en lugar de ponerse a maratonear series, se ponen a leer."*

Mostró la página de Spring, con todos los proyectos que tienen soporte: Spring Boot, Cloud, Data, Integration, Batch y Security. Dijo que eventualmente se usarán Data y, "definitivamente", Security. Tiene además herramientas de desarrollo: **Spring Tools**, con el plugin para Visual Studio Code, y **STS**, que se puede bajar como programa independiente o como complemento de Eclipse.

Crear el proyecto desde `start.spring.io` (minuto 57:00)

1. Abres el Initializr en el navegador.
2. Eliges: proyecto **Maven**, lenguaje **Java**, y una versión de Spring Boot "generalmente aceptada" (no *snapshot*).
3. Llenas los datos del proyecto: package y versión de Java (él pone 26).
4. Agregas las dependencias: DevTools, Lombok, Web, Data JPA, PostgreSQL o MySQL, Validation, Security.
5. Das **Generate** y se descarga un **.zip**. Lo desempaquetas, lo abres y lo corres.

— ¿Qué es una versión *snapshot* ?

Una versión en desarrollo, todavía no publicada como estable. Cambia sin aviso. Para trabajar conviene la última estable, que es a la que se refiere "generalmente aceptada".

Dudas finales y cierre

Los últimos minutos fueron el profesor caminando entre las computadoras. Salieron tres fórmulas de las tareas, y una revisión de que las respuestas se vieran bien.

Alumno: *"Profe, ¿cómo era la fórmula de Fahrenheit a Celsius?"*

Profesor: *"(fahrenheit - 32) * 5 / 9."* **Alumno:** *"¿Y de Celsius a Kelvin?"*

Profesor: *"celsius + 273.15."* **Alumno:** *"¿Y de Kelvin a Fahrenheit?"*

Profesor: *"(kelvin - 273.15) * 9 / 5 + 32."*

Entre 61:30 y 64:47 se acercó a revisar que las respuestas JSON y el código 200 se vieran bien en la herramienta gráfica de cada quien. Hay una frase suelta ("Ya aprovechando que tienen que hacer los cursos... Está bien. Con calma") que no queda clara: probablemente habla de algo externo a la materia.

"Listo, señores. Mañana continuamos en el Laboratorio 3."

CÓDIGO DE LA CLASE

El proyecto de corrido

Los tres archivos que se escribieron o tocaron, completos y en el estado en que terminó la clase. El controlador ya trae los dos métodos hechos y un espacio para los cuatro que faltan.

`application.properties`

```
# spring.application.name lo genera el asistente con el nombre del proyecto
server.port=8082
```

`dto/ConversionResponse.java`

```
package com.ipn.mx.dto;

public record ConversionResponse(
    Double valorOriginal,
    String unidadOriginal,
    Double valorRespuesta,
    String unidadRespuesta
) {}
```

`controller/TemperaturaController.java`

```

package com.ipn.mx.controller;

import com.ipn.mx.dto.ConversionResponse;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RequestParam;
import org.springframework.web.bind.annotation.RestController;

@RestController
@RequestMapping("/api/temperatura")
public class TemperaturaController {

    @GetMapping("/convertir-a-fahrenheit")
    public ResponseEntity<ConversionResponse> celsiusToFahrenheit(
        @RequestParam(name = "valor") Double celsius
    ) {
        Double fahrenheit = (celsius * 9 / 5) + 32;
        ConversionResponse response = new ConversionResponse(celsius, "°C", fahrenheit, "
        return ResponseEntity.ok(response);
    }

    @GetMapping("/convertir-celsius-a-kelvin")
    public ResponseEntity<ConversionResponse> celsiusToKelvin(
        @RequestParam(name = "valor") Double celsius
    ) {
        Double kelvin = celsius + 273.15;
        ConversionResponse response = new ConversionResponse(celsius, "°C", kelvin, "K");
        return ResponseEntity.ok(response);
    }

    // Tarea: F a C, F a K, K a C y K a F
}

```

– ¿En qué se diferencia esta versión de lo que quedó en el pizarrón?

Son cabos sueltos del ritmo de la clase, no correcciones a lo que dijo:

- **Imports.** No los mostró en la transcripción; los agregué para que compile. Los del `RestController` y `RequestMapping` sí aparecen ahí, el resto los pone el IDE solo.
- **Nombre del endpoint de Kelvin.** Aquí va `convertir-celsius-a-kelvin`, el que probó en vivo, y no `convertir-a-kelvin`, que trae el bloque de código de la transcripción.
- **Nombres asimétricos.** El de Fahrenheit se queda sin "celsius" en el nombre. Lo dejé literal; tú decides cómo nombrar los cuatro que faltan.
- **Formato.** Solo salto de línea en los parámetros para que se lea en pantalla angosta.
- **Comentario de tarea.** El `// Tarea` es mío, para marcar dónde van los otros cuatro.

Checklist de la clase 8

Se guarda en tu navegador. Nada se sube a ningún lado ni se comparte con nadie más.

0 / 4

☐ **Tarea 1 · terminar el conversor de temperaturas**

Faltan cuatro: Fahrenheit a Celsius, Fahrenheit a Kelvin, Kelvin a Celsius y Kelvin a Fahrenheit. Celsius a Fahrenheit y Celsius a Kelvin los hizo él. Valores de prueba: 10 °C a 50 °F, 20 °C a 68 °F, 20 °C a 293.15 K.

☐ **Pendiente administrativo · laboratorio del jueves 10 en el Laboratorio 3**

Ahí se revisan los demás endpoints. Es un cambio de aula: "los jueves es en el 3". Lo dijo dos veces, a media clase y al cierre.

☐ **Por confirmar · qué vence "la fecha del 17 de septiembre"**

La mencionó al hablar de altas y bajas, sin aclarar. Pregúntale si te afecta. No confundir con la entrega de la investigación de HTTP.

☐ **Recomendación · leer HELP.md y la documentación de Spring**

No es tarea: dijo que vale la pena revisar el HELP.md del proyecto y que la documentación de Spring sea "su biblia".